

Achievement numbers of four small polyominoes

Jan Philipp Harries*

October 8, 2026

Abstract

In Harary’s achievement game for a polyomino P two players alternately claim cells of an $n \times n$ board, and the first player tries to own a congruent copy of P . Gardner’s tables give, for each polyomino that the first player can achieve, a board number b (the side of the smallest square board on which the first player wins) and a move number m (the number of the first player’s moves needed there); OEIS A380597 and A380598 reproduce the values of Gardner’s 2001 book. For the I-tetromino and the L-, Y- and N-pentominoes we determine (b, m) in the Maker–Breaker (weak) game as $(6, 6)$, $(6, 7)$, $(6, 6)$ and $(5, 6)$, and in the strong game, in which a player who completes a copy first wins, as $(7, 7)$, $(7, 9)$, $(6, 6)$ and $(5, 6)$. Hence the tabulated pairs $(7, 9)$ for the Y-pentomino and the board number 6 for the N-pentomino are wrong in both games; for the I-tetromino, tabulated $(7, 8)$, and the L-pentomino, tabulated $(7, 10)$, both values are wrong in the weak game, and in the strong game the board number 7 is correct and the move number is wrong. Every statement is proved by an explicit strategy that short programs check exhaustively; for the strong game on 6×6 , and for the move numbers on 7×7 of the I-tetromino and the L-pentomino, these are strategies of the second player that block, complete copies of their own, and end lines in pairing strategies. The strategies and programs are in the ancillary files.

1 Introduction

Harary’s generalization of tic-tac-toe, popularized by Gardner in 1979 [3], fixes a polyomino (an “animal”) P and a square board. Two players alternately claim free cells, and a player who owns all cells of a copy of P —rotations and reflections allowed—wins. By strategy stealing the second player cannot win with correct play, so the game is a first-player win or a draw. Gardner describes this *strong* game: the first player to complete the animal wins, and for a polyomino that cannot be achieved the second player “can always force a draw” [5, p. 497]. Later work, for example [6, 12], studies the *weak* or Maker–Breaker version, in which the second player only blocks.

For every polyomino that the first player can achieve, Gardner tabulates the *board number* b , the side of the smallest square board on which the first player can force a win, and the *move number* m , the number of the first player’s moves needed to win on that board. The tables in Gardner’s 1992 and 2001 books [4, 5] (in [5]: chapter 37, Figures 37.2 and 37.3) give no proofs, and Gardner invites the reader to prove the values [5, p. 497]. They differ in several entries from the 1979 column as quoted in [14], whose table had already been questioned in [13]. *Winning Ways* [1] apparently tabulates a different pair, the least number of moves on an unbounded board and the least board on which that number suffices [13]; we do not compare with it. The OEIS entries A380597 and A380598 [10, 11], by P. von Brömssen (2025), list b and m for the free polyominoes in the order of

*ellamind GmbH, Bremen, Germany. E-mail address: jan@ellamind.com

A246521 [9]; for every polyomino that the first player can achieve they agree with Gardner's 2001 tables. Neither entry says which game is meant.

This note settles the board and move numbers of four of these polyominoes in both games (Section 3). The other entries of size at most five agree with the tables in the weak game (Remark 8). The proofs are finite: each statement is reduced to an explicit strategy for one of the players, written as a certificate file and checked exhaustively by programs that share no code with the search that found it (Section 4).

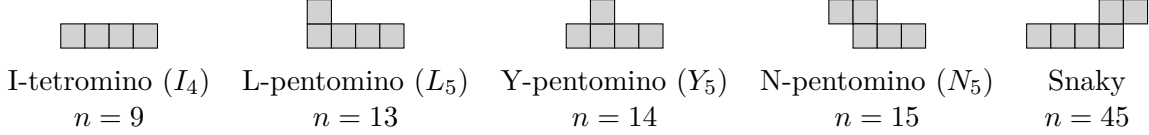


Figure 1: The polyominoes of this note and their indices n in A380597/A380598 (the polyomino with code A246521($n + 1$)).

2 Games, board numbers and move numbers

Let $[n] = \{0, \dots, n - 1\}$ and $B_n = [n]^2$. A *placement* of a polyomino $P \subset \mathbb{Z}^2$ on B_n is a set $g(P) \subseteq B_n$, where g is one of the eight symmetries of the square lattice followed by a translation; $\mathcal{P}_n(P)$ is the set of placements. Two players, X (first) and O , alternately claim free cells of B_n , starting with X .

Definition 1. In the *weak game* X (Maker) wins as soon as he owns a placement; O (Breaker) wins if this never happens. In the *strong game* a player who owns a placement before the other does wins; if the board fills first, the game is a draw. For $d \geq 1$, X *wins within d moves* if X has a strategy such that against every play of O he owns a placement after at most d of his own moves (in the strong game: and O owns no placement before that moment). The *board number* $b^w(P)$ is the least n such that X wins the weak game on B_n within some d , and the *move number* $m^w(P)$ is the least d such that X wins the weak game on $B_{b^w(P)}$ within d moves. $b^s(P)$ and $m^s(P)$ are defined in the same way with the strong game.

This is the convention of Gardner's tables: for the I-tromino the board number is 4 and the move number 3 [5, p. 494]. The following two facts are standard; we include proofs because the lower bounds below rest on them.

Lemma 2. Fix P , n and d .

- (a) If O has a strategy in the weak game on B_n against which X owns no placement after any of his first d moves, then X does not win the strong game on B_n within d moves.
- (b) If X wins the strong game on B_n within d moves, he wins the weak game on B_n within d moves.

Proof. (a) In the strong game O follows the blocking strategy. As long as the strong game goes on, the positions are positions of the weak game under that strategy, so X owns no placement after any of his first d moves; if the strong game stops earlier, it stops with a draw or a win for O . (b) X uses the same strategy. Until X completes a placement the two games pass through the same positions, and the strong strategy completes one within d moves. $\square$

Lemma 3. *If $k \leq n$ and X wins the weak game on B_k within d moves, then X wins the weak game on B_n within d moves. Consequently a blocking strategy for O against d moves (or against any number of moves) on B_n yields one on every B_k with $k \leq n$, since finite games of perfect information are determined.*

Proof. Let σ be the winning strategy on $B_k \subseteq B_n$. X plays σ in an imagined game on B_k in which X owns the same cells as in reality and O owns a superset of O 's real cells in B_k . A real move of O on a cell of B_k that is free in the imagined game is also O 's imagined move; any other move of O is replaced in the imagined game by a move on an arbitrary free cell of B_k . Such a cell exists as long as X has not won, since σ wins before the imagined board is full. Every move of σ is free in the imagined game and therefore in reality, and every placement on B_k is a placement on B_n , so X completes one within d moves. $\square$

Lemma 2 gives $b^w \leq b^s$, and $m^w \leq m^s$ when $b^w = b^s$. Lemmas 2(a) and 3 turn one blocking strategy on B_{b-1} into the lower bound b on the board number in both games. We use no analogue of Lemma 3 for the strong game: a strategy of O in the strong game on B_n that is not a blocking strategy is used only on B_n itself.

3 Results

Proved below means that the statement follows from Lemmas 2 and 3, Propositions 11 and 12 and from certificates listed in Table 1, each accepted by every checker named there (Section 4). *Computed* means a verdict of a search program for which no certificate exists.

Theorem 4 (weak game). (b^w, m^w) equals $(6, 6)$ for I_4 , $(6, 7)$ for L_5 , $(6, 6)$ for Y_5 and $(5, 6)$ for N_5 .

Theorem 5 (strong game). (b^s, m^s) equals $(7, 7)$ for I_4 , $(7, 9)$ for L_5 , $(6, 6)$ for Y_5 and $(5, 6)$ for N_5 . In particular, for I_4 and for L_5 the first player does not win the strong game on any board B_n with $n \leq 6$; with correct play of the second player such a game ends in a draw or in a win for the second player.

Proof of Theorems 4 and 5. Take I_4 in the weak game. The Breaker certificate for the full game on B_5 and Lemma 3 give $b^w \geq 6$; the Maker certificate on B_6 with budget 6 gives $b^w \leq 6$ and $m^w \leq 6$; the Breaker certificate against 5 moves on B_6 gives $m^w \geq 6$. The other shapes are the same with the files of Table 1. In the strong game the two Breaker certificates of a shape give, by Lemmas 2(a) and 3, $b^s \geq b^w$ and, for Y_5 and N_5 , that X does not win on B_{b^w} within $m^w - 1$ moves; the strong strategies give the matching upper bounds. For I_4 and L_5 : on B_n with $n \leq 5$, X does not win the strong game by the Breaker certificate on B_5 and Lemmas 2(a) and 3; on B_6 he does not win by the O -strategy certificate with budget 18 and Proposition 12, since on B_6 the first player makes at most 18 moves; on B_7 he wins within 7 (I_4) and 9 (L_5) moves by the strong strategies, and not within 6 and 8 moves by the O -strategy certificates with these budgets. Hence $b^s = 7$ and $m^s = 7$, respectively 9. $\square$

The smallest certificate is a single pairing: on B_4 , cells (x, y) , the six pairs $\{(x, 0), (x, 1)\}$ for $x \in [4]$ and $\{(0, 2), (1, 2)\}$, $\{(2, 2), (3, 2)\}$ meet every placement of N_5 in a full pair, so O , answering each move of X on a paired cell with its partner, prevents N_5 on B_4 (and, by Lemma 3, on smaller boards).

shape	statement	file in anc/certs/	size	checked by
I_4	O blocks on B_5	n9_b5_full	10,130	A, B
	X wins weak game on B_6 within 6	n9_b6_moves	2,615	A, R
	O blocks 5 moves on B_6	n9_b6_lt6	2,227	A, B
	X wins strong game on B_7 within 7	strong_n9_b7_d7	333,113	S, T
	X does not win strong game on B_6 nor on B_7 within 6	i4_b6_full i4_b7_K6	4,154,214 21,474	C, D, E C, D, E
L_5	O blocks on B_5	n13_b5_full	321	A, B
	X wins weak game on B_6 within 7	n13_b6_moves	11,075	A, R
	O blocks 6 moves on B_6	n13_b6_lt7	981	A, B
	X wins strong game on B_7 within 9	strong_n13_b7_d9	948,483	S, T
	X does not win strong game on B_6 nor on B_7 within 8	l5_b6_full l5_b7_K8	471,778 884,056	C, D, E C, D, E
Y_5	O blocks on B_5	n14_b5_full	293	A, B
	X wins weak game on B_6 within 6	n14_b6_moves	3,913	A, R
	O blocks 5 moves on B_6	n14_b6_lt6	213	A, B
	X wins strong game on B_6 within 6	strong_n14_b6_d6	1,023,521	S, T
N_5	O blocks on B_4	n15_b4_full	1	A, B
	X wins weak game on B_5 within 6	n15_b5_moves	3,882	A, R
	O blocks 5 moves on B_5	n15_b5_lt6	84	A, B
	X wins strong game on B_5 within 6	strong_n15_b5_d6	253,534	S, T

Table 1: Certificates behind Theorems 4 and 5. “Blocks” means a Breaker strategy in the weak game, against the full game or against the stated number of Maker moves. “Does not win” is shown by a strategy of O in the strong game (Section 4); the budget of the B_6 certificates is 18, the whole game. Size: certificate nodes; for strong strategies of X , positions with X to move; for strategies of O , table lines, i.e. positions with O to move plus pairing leaves, each up to the eight symmetries of the board. The four strategies of O are stored gzipped with sizes 16.2, 0.10, 1.8 and 3.7 MB. Checkers: A `check_cert.py`, B `fresh_checker/check_breaker2.py`, R `play_cert.py` (exhaustive concrete replay), S `check_strong.py`, T `fresh_checker/check_strong2.py`, C `strong/check6.py`, D `strong/check6b.cpp`, E `strong/chk.py`.

Corollary 6. *In both games the values $(b, m) = (7, 9)$ tabulated for Y_5 are wrong (both are 6), and the board number 6 tabulated for N_5 is wrong (it is 5); the tabulated move number 6 of N_5 is correct. For I_4 , tabulated $(7, 8)$, and L_5 , tabulated $(7, 10)$, both values are wrong in the weak game. In the strong game the board number 7 is correct for both, and the move numbers are wrong: they are 7 for I_4 and 9 for L_5 .*

In particular the example line of A380597, that the I-tetromino is a draw on square boards of side less than 7, is true in the strong game (Theorem 5) and false in the weak game, where Maker wins on 6×6 .

The 1979 board number 6 for I_4 , L_5 and Y_5 agrees with the weak game; in the strong game it holds for Y_5 and does not hold for I_4 and L_5 , whose strong board number is 7, as in the 1992/2001 tables. The 1979 pairs $(6, 8)$ for I_4 and $(6, 9)$ for L_5 match neither game: on 6×6 the first player wins the weak game within 6 and 7 moves and does not win the strong game. We know of no source that gives board number 5 for N_5 . For I_4 a weak-game win on 7×7 within 7 moves was given earlier by Earnest and Mathias [2] (cited in [14]); Theorem 4 improves it to 6×6 within 6 moves.

Remark 7 (computed consistency checks). The following solver verdicts are not used in any proof; they agree with the certified values. (i) For I_4 and L_5 on B_7 , the first and the second solver of

shape	n	Gardner 1979*	Gardner 1992/2001, OEIS	weak game	strong game
I_4	9	6 / 8	7 / 8	6 / 6	7 / 7
L_5	13	6 / 9	7 / 10	6 / 7	7 / 9
Y_5	14	6 / –	7 / 9	6 / 6	6 / 6
N_5	15	–	6 / 6	5 / 6	5 / 6

Table 2: Board number / move number. *Not seen by the author; values as quoted in [14], which annotates only some 1979 entries (“–”: no 1979 value quoted there). The weak and strong columns are Theorems 4 and 5.

Section 4 both find a win for X in the strong game within 7 and 9 moves and none within 6 and 8 moves. On B_6 the second solver finds no win for X within 11 moves for I_4 (151 s) and within 13 moves for L_5 (878 s); the first solver reaches 8 and 10. The second solver does not decide the full 6×6 game within 1700 s; it is decided by the O -strategy certificates of Table 1, which the third solver `strong/solver_strong.cpp` produced. (ii) As controls, the second solver gives (5, 4) for the T-tetromino, (4, 4) for the L-tetromino and (3, 5) for the S-tetromino in both games, as in the tables.

Remark 8 (checked with one checker). With the first solver, Breaker and Maker certificates of the kind in Table 1 were also generated for all entries $n \leq 21$ (polyominoes of size at most five), and each is accepted by `check_cert.py`; no second checker was run on them. They reproduce A380597 and A380598 in the weak game except at $n = 9, 13, 14, 15$, and for the entries with value 0 they show that X does not win on boards up to 8×8 . These certificates are not in the ancillary files; `validate_small.py` regenerates and checks them in under a minute.

Remark 9 (Snaky). A380597 lists Snaky, $n = 45$ (Figure 1), as its only unknown value. Halupczok and Schlage-Puchta proved that Breaker wins the weak game on 8×8 , with a strategy found by computer [6, Theorem 1(3), Section 2]. Our solver produces Breaker certificates for this board, one for each of the 10 classes of Maker’s first cell under the symmetries of the board; each passes `check_cert.py` (the largest has 1,126,032 nodes). This reproduces the result as a checked certificate. By Lemmas 2 and 3, if Snaky is achievable on some square board in either game, its board number is at least 9. These certificates (191 MB) are not in the ancillary files; `split_run.py` regenerates and checks them in about 40 minutes with three processes. Whether Snaky can be achieved on a finite board in the strong game is open, and its board number is unknown in both games; for the weak game a first-player strategy on 17×17 has been announced in a preprint [8], which this note does not use.

4 Method

Representation and search. Cells are numbered $y \cdot W + x$; positions are pairs of bitmasks (M, B) , and placements are enumerated from the shape by the eight lattice symmetries and all translations. The search program `solver.cpp` answers “does X win within d moves?” by depth-first AND/OR search with a transposition table keyed by 64-bit hashes, canonical under the symmetries of the board, and with pruning rules listed in the ancillary README. A hash collision or a pruning error could make a verdict wrong; therefore nothing is called proved on the strength of a verdict. For proved results the solver exports a certificate, and the certificate is checked by programs that use no hashing and no pruning beyond the rules proved below.

Breaker certificates. Let d be Maker’s remaining number of moves ($d = \infty$ for the full game). A placement p is d -completable at (M, B) if $p \cap B = \emptyset$ and $|p \setminus M| \leq d$; a free cell is d -relevant if it lies in a d -completable placement. A certificate is a directed acyclic graph whose nodes are evaluated along every play from the empty board, with a *virtual* position (M, B, d) attached. The checker `check_cert.py` requires at every reached node that M contains no placement, and:

m (Maker to move, $d \geq 1$): a child for every d -relevant cell c , at $(M \cup \{c\}, B, d - 1)$; if some free cell is not d -relevant, also a *pass child* at $(M, B, d - 1)$;

b (Breaker to move): a free d -relevant cell c and a child at $(M, B \cup \{c\}, d)$;

Z, D (leaves): $d = 0$, or no d -relevant cell exists;

P (leaf, Maker to move): pairwise disjoint pairs of free cells such that every d -completable placement contains one of the pairs.

Lemma 10. *If p is not d -completable at some position, it is not completable for the then remaining budget at any later position, also when Maker’s moves are replaced by passes.*

Proof. A move of O only enlarges B . A move of X lowers $|p \setminus M|$ by at most one and d by exactly one; a pass lowers only d . So $|p \setminus M| > d$ persists. $\square$

Proposition 11. *If `check_cert.py` accepts a Breaker certificate with budget d_0 , then O has a strategy in the weak game against which X owns no placement after any of his first d_0 moves.*

Proof. O walks the certificate. When X plays a d -relevant cell, O moves to its child; when X plays another cell g , O moves to the pass child and calls g a *ghost*: the virtual position omits ghosts from M and otherwise equals the real one, with the same budget. At **b** nodes O plays the given cell.

(1) *No placement through a ghost is ever completable in reality.* When g was played, every placement $p \ni g$ with $p \cap B = \emptyset$ had $|p \setminus M_{\text{virt}}| > d$. If p contains no earlier ghost, its real and virtual counts agree, so p was not completable in reality, and it stays so by Lemma 10; placements through earlier ghosts are covered by induction. In the virtual game a ghost is a pass, so by Lemma 10 it never becomes relevant there: O is never told to play a ghost, and the cells O plays are free in reality.

(2) By (1), every placement that is completable in reality avoids the ghosts and is completable in the virtual position. Hence X owns no real placement while the virtual M contains none (checked at every node); at **Z** the budget is used up; at **D** nothing is completable; at **P** every placement that X could still complete within the budget contains a pair of really free, non-ghost cells, and from then on O answers each move of X in a pair with its partner (by Lemma 10 no other placement becomes completable later). $\square$

At a **P** leaf a placement with one missing cell cannot contain a pair of free cells, so the condition also excludes immediate threats. Pairing strategies are the classical way to show that a polyomino cannot be achieved (see [13, 6]); here only the placements completable within the budget need to be covered.

Maker certificates. A Maker certificate gives X ’s cell at each node and, at O ’s turn, explicit children for some replies and a default child for the rest; `check_cert.py` accepts a default only for replies outside the default subtree’s envelope (the cells X claims there and the cells of the placements he completes there). We do not rely on this rule: `play_cert.py` follows each Maker certificate as a strategy against *every* legal reply of O at every reached position, memoized on (node, M , B), and requires a real placement within the budget. The four Maker certificates of Table 1 pass this replay with 174,766, 702,984, 288,636 and 89,274 distinct states.

Strong-game strategies. A strong certificate is a table from positions with X to move to X 's move. `check_strong.py` starts from the empty board, plays the table's move, stops if X owns a placement, and otherwise fails if the budget is used up or the board is full; it then tries every free cell as O 's reply, fails if O owns a placement, and continues. No pruning is used, so acceptance is a complete proof that X wins within the budget. The tables were exported with every reply of O enumerated; the four tables of Table 1 have 253,534 to 1,023,521 positions, and every table entry is reached.

Strategies of O in the strong game. To show that X does not win the strong game on B_n within K moves we give a strategy of O . The game ends when O completes a placement first, so O may attack as well as block; a pure blocking strategy, as used for the weak game, does not exist on B_6 for I_4 and L_5 , because Maker wins the weak game there. For a position (X, O) let $r = K - |X|$ be the number of moves X has left, and call a placement p *r-live* if $p \cap O = \emptyset$ and $|p \setminus X| \leq r$.

A certificate of kind `ostrategy` (format in the ancillary README) consists of a header (board, shape, kind, budget K) and a table indexed by *canonical keys*: the key of (X, O) is the least pair $(g(X), g(O))$, compared as integers via bitmasks, over the eight symmetries g of the board. An entry `O` gives O 's move at a position with O to move; an entry `P` gives pairs of cells at a position with X to move. Cells are stored in the frame of the key; at a real position Q the checker picks a symmetry g with $g(Q)$ equal to the key and maps a stored cell c to $g^{-1}(c)$. The checker walks the game from the empty board. At a position with X to move it tries *every* free cell and fails if the move completes a placement of X ; at a position with O to move it requires a table entry whose cell is free. A line of play ends, and is accepted, exactly at these positions:

- (a) O 's move completes a placement of O ;
- (b) X has made K moves ($r = 0$);
- (c) the board is full;
- (d) no placement is r -live (*hopeless*);
- (e) X is to move and the table has a `P` entry whose pairs, mapped to the real board, consist of two distinct free cells each, are pairwise disjoint, and every r -live placement contains both cells of some pair (*pairing leaf*).

Tests (b)–(d) are made at positions of both kinds. No other pruning is used: X 's moves are never restricted. The three checkers test these conditions in different orders; since each accepted leaf is sound on its own, the order does not matter.

Proposition 12. *If one of the three checkers in `strong/` (`check6.py`, `check6b.cpp`, `chk.py`) accepts an `ostrategy` certificate with budget K on B_n , then O has a strategy in the strong game on B_n against which X never owns a placement after one of his first K moves unless O owned a placement before. In particular X does not win the strong game on B_n within K moves, and not at all if $K \geq \lceil n^2/2 \rceil$.*

Proof. Symmetry. `check6.py` and `chk.py` verify that the set $\mathcal{P}_n(P)$ is mapped onto itself by the eight symmetries of the board, so the conditions checked at a position—legality of a move, completion of a placement, (b)–(e)—hold at Q for a move m if and only if they hold at $h(Q)$ for $h(m)$, for every symmetry h , and Q and $h(Q)$ have the same key. `check6b.cpp` needs no such argument: it memoizes on exact positions and verifies every real position that occurs, with the move it plays there. `check6.py` memoizes on keys: when it meets a position Q' whose key it has

verified at Q , then $Q' = h(Q)$ for some symmetry h , and the verified tree below Q maps under h to a verified tree below Q' . `chk.py` verifies the canonical positions themselves and canonicalizes every child, which is the same argument. In all three cases O 's strategy is: at a real position Q whose key carries an 0 entry c , play $g^{-1}(c)$ for a symmetry g with $g(Q)$ equal to the key; different choices of g differ by a symmetry fixing Q and give moves with isomorphic verified subtrees.

Internal positions. Along any play that stays inside the verified tree, no move of X completes a placement, since every free cell was tried.

Leaves. At (a) O owns a placement and X none, and the game is over. At (c) the game is a draw. At (b) X has made K moves without owning a placement. At (d) and (e) let O continue as follows: if X claims a cell of a pair whose partner is free, O claims the partner; otherwise O claims any free cell. (At (d) there are no pairs.) A placement p that X owns after at most r further moves is disjoint from O 's cells at all later times, hence at the leaf, and needs at most r further cells, so it was r -live at the leaf. At (d) no such placement exists. At (e) it contains a pair $\{u, v\}$ of cells that were free at the leaf. The statement “every pair is entirely free or contains a cell of O ” holds at the leaf and is preserved: when X claims a cell of an entirely free pair, O claims its partner at once (the pairs are disjoint, so one answer suffices), and a pair containing a cell of O keeps it. Hence X never owns both u and v , and never owns p . If O completes a placement or the board fills before, the game ends earlier, without a win for X .

Finally, X makes at most $\lceil n^2/2 \rceil$ moves on B_n , so with $K \geq \lceil n^2/2 \rceil$ the budget is no restriction. $\square$

The pairing rule (e) covers only placements completable within the remaining budget, as in Proposition 11; it is applied only at positions where it is checked directly, so no analogue of the pass children of Breaker certificates is needed. For the 6×6 certificates the budget is 18, the whole game, and (b) never occurs.

The O strategies of Table 1 were found by a third solver, `strong/solver_strong.cpp`: depth-first proof-number search [7] with a transposition table keyed by the full canonical position (two 64-bit words, so without hash collisions), immediate wins, double threats and forced blocks for both players, the leaves (d) and (e) (pairings found by a bounded backtracking search), and a move restriction to cells in placements still completable by either player. That restriction is not proved for the strong game and is not used by the checkers: the export enumerates every free cell at each position with X to move and solves every position the search had pruned. After an opening move of X on the border of B_6 , O 's answer already leads to a pairing leaf; after an interior opening the strategy also uses threats of its own, and 3,331,388 of the 3,880,764 0 entries of the I_4 certificate complete a placement of O . `strong/check6.py` and `strong/chk.py` report the counts of the leaf kinds and of unused table entries; there are no unused entries in any of the four certificates. Expanded to exact positions, the I_4 certificate on B_6 covers 2,071,013 positions with X to move and 15,939,736 with O to move.

Independent programs. The directory `fresh_checker/` holds a second checker for Breaker certificates, `check_breaker2.py`, and for strong strategies, `check_strong2.py`, written independently, from the format description alone, with their own placement enumeration; they accept every Breaker certificate and every strategy of X in Table 1. A second solver, `fresh_solver/ttt.cpp`, was written independently, from the rules of the game alone, with its own pruning; on boards up to 4×4 it agrees with an unpruned minimax (`naive.cpp`) and on 5×5 with an unpruned budgeted search (`naive_b.cpp`) for budgets 3 to 5, in both games. It reproduces every value of Theorem 4, the values (6,6) and (5,6) of Theorem 5 and the 7×7 verdicts of Remark 7(i). The

script `fresh_checker/index_own.py` enumerates the polyominoes of size at most five, recomputes A246521 and confirms the indices 9, 13, 14, 15.

The three checkers for strategies of O share no code with `strong/solver_strong.cpp` or with each other. `strong/check6.py` (Python) memoizes on canonical keys; `strong/check6b.cpp` (C++) memoizes on exact positions and has its own placement enumeration and symmetry formulas; `strong/chk.py` (Python) was written from the format description alone, without reading the solver or the other two checkers. Each takes the claimed board, shape, kind and budget on the command line and fails if the certificate header differs; each prints the number of placements (36 and 120 for I_4 and L_5 on B_6 , 56 and 192 on B_7 , the counts $2n(n-3)$ and $8(n-3)(n-1)$). They report the same numbers of O and P table lines. In further tests, corrupted certificates (a dropped O or P entry, a move onto an occupied cell or off the board, a pair dropped, a pair on an occupied cell, overlapping pairs, the budget raised by one) were rejected by all three checkers. On boards up to 4×4 , and on 5×5 for budgets up to 6, the third solver's verdicts agree with an unpruned minimax search for 14 polyominoes, and the certificates it exports there pass `strong/check6.py` and `strong/check6b.cpp`. The X strategies of Table 1, converted to canonical keys, pass these two checkers as well. The scripts of these tests are not part of the ancillary files.

What is trusted. The proofs depend on: the checkers (a few hundred lines each, available in two separately written versions for Breaker certificates and strategies of X in the strong game, and in three for strategies of O) and their placement enumeration; the proof rules of Propositions 11 and 12; the replay `play_cert.py` for the four weak Maker wins, which for Y_5 and N_5 also follow from the strong strategies by Lemma 2(b); the Python interpreter; and, for `strong/check6b.cpp`, the C++ compiler. They do not depend on any of the three solvers.

5 Reproduction

The ancillary directory `anc/` contains the certificates of Table 1, the programs named above, a manifest of SHA-256 sums and a README with every command and its expected output. With `uv` and a C++17 compiler:

```
cd anc
shasum -a 256 -c MANIFEST
bash verify_all.sh          # 44 checks, about 4 minutes; ends "ALL 44 CHECKS PASSED"

clang++ -O2 -std=c++17 -o solver solver.cpp          # regenerate, e.g. N5 strong:
./solver --board 5 --shape "1,0;0,1;2,0;1,1;3,0" --game strong --dmin 6 --dmax 6 \
    --tt-mb 512 --cert s15.txt && uv run python check_strong.py s15.txt
bash regenerate_weak.sh    # the 12 weak certificates, checked; a few seconds

cd fresh_solver && clang++ -O2 -std=c++17 -o ttt ttt.cpp
./ttt Y5 6 strong 7        # K=5 no_win_within_K, K=6 P1_WINS
./ttt L5 7 strong 10       # K=8 no_win_within_K, K=9 P1_WINS

cd ../strong               # strategies of O; note -fno-vectorize
clang++ -O2 -fno-vectorize -std=c++17 -o solver_strong solver_strong.cpp
./solver_strong --board 6 --shape "0,0;1,0;2,0;3,0" --K 18 --tt-log2 23 \
    --max-rss-mb 3800 --cert i4_b6_full1.txt          # DONE x_cannot_win ...
uv run python chk.py i4_b6_full1.txt --board 6 --shape "0,0;1,0;2,0;3,0" \
    --kind ostrategy --budget 18                      # PASS
```

Weak-game and strong X certificates regenerated with `solver.cpp` may differ in size from the included ones, which were exported by an earlier version of that solver; every regenerated certificate passes the checkers. The export of `strong/solver_strong.cpp` is deterministic and reproduces the included O strategies byte for byte. Apple clang 21 at plain `-O2` miscompiles a range-based loop of this solver’s placement generator (in the loop vectorizer; the wrong placement set is caught by the solver’s own symmetry self-check). The loop is written with indices, and the solver and `strong/check6b.cpp` are built with `-O2 -fno-vectorize` (with GCC, `-O2 -fno-tree-vectorize`).

Use of AI tools

The programs in the ancillary files—all three solvers, all checkers and the driver scripts—and drafts of this note were developed in sessions assisted by large-language-model coding tools. The author checked the arguments and the proof-critical outputs and takes responsibility for the note. The proofs do not depend on the correctness of the search programs: they rest on the lemmas proved above and on certificates that anyone can recheck with the included checkers. Agreement between separately written, AI-assisted programs is an error filter, not an independent reproduction by other authors.

References

- [1] E. R. Berlekamp, J. H. Conway and R. K. Guy, *Winning Ways for your Mathematical Plays*, Vol. 2, Academic Press, London, 1982, pp. 684–685. (Not seen by the author; cited after [13, 14].)
- [2] M. Earnest and D. Mathias, answer to “Harary’s generalized Tic-Tac-Toe; Winning strategy for Skinny on a 7×7 board?”, Puzzling Stack Exchange, January 15, 2022, <https://puzzling.stackexchange.com/a/114453/3896>. (Not seen by the author; cited after [14].)
- [3] M. Gardner, Mathematical Games: In which players of ticktacktoe are taught to hunt bigger game, *Scientific American* **240**(4) (April 1979). (Not seen by the author; values quoted after [14].)
- [4] M. Gardner, *Fractal Music, Hypercards and More...: Mathematical Recreations from Scientific American Magazine*, W. H. Freeman, New York, 1992. (Table seen as reproduced at <https://mathlair.allfunandgames.ca/ticktactoe.php>.)
- [5] M. Gardner, Harary’s generalized ticktacktoe, Chapter 37 in *The Colossal Book of Mathematics*, W. W. Norton, New York, 2001, pp. 493–503.
- [6] I. Halupczok and J.-C. Schlage-Puchta, Achieving Snaky, *Integers* **7** (2007), #G02.
- [7] A. Nagai, *Df-pn Algorithm for Searching AND/OR Trees and Its Applications*, PhD thesis, University of Tokyo, 2002.
- [8] OpenAI, Snaky in 21 Maker moves, preprint, September 25, 2026, <https://github.com/openai/math/blob/main/preprints/Snaky-in-21-Maker-moves-September-25-2026/article.pdf>.

- [9] OEIS Foundation Inc., The On-Line Encyclopedia of Integer Sequences, sequence A246521, <https://oeis.org/A246521>, accessed October 7, 2026.
- [10] P. von Brömssen, Sequence A380597, in *The On-Line Encyclopedia of Integer Sequences*, <https://oeis.org/A380597>, accessed October 7, 2026.
- [11] P. von Brömssen, Sequence A380598, in *The On-Line Encyclopedia of Integer Sequences*, <https://oeis.org/A380598>, accessed October 7, 2026.
- [12] N. Sieben, Snaky is a 41-dimensional winner, *Integers* **4** (2004), #G5.
- [13] C. Thompson, Harary’s animal-achievement game – some questions, posting to `sci.math`, October 14, 1996, archived at <https://ics.uci.edu/~eppstein/junkyard/animal-game.html>.
- [14] Wikipedia, Harary’s generalized tic-tac-toe, https://en.wikipedia.org/wiki/Harary's_generalized_tic-tac-toe, accessed October 7, 2026.